

rxqvnbo5a

January 16, 2025

```
[1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

0.0.1 Read Dataset

```
[2]: data = pd.read_csv("cardekho.csv")
data.head()
```

```
[2]:
```

		name	year	selling_price	km_driven	fuel	\
0		Maruti Swift Dzire VDI	2014	450000	145500	Diesel	
1		Skoda Rapid 1.5 TDI Ambition	2014	370000	120000	Diesel	
2		Honda City 2017-2020 EXi	2006	158000	140000	Petrol	
3		Hyundai i20 Sportz Diesel	2010	225000	127000	Diesel	
4		Maruti Swift VXi BSIII	2007	130000	120000	Petrol	

	seller_type	transmission	owner	mileage(km/ltr/kg)	engine	\
0	Individual	Manual	First Owner	23.40	1248.0	
1	Individual	Manual	Second Owner	21.14	1498.0	
2	Individual	Manual	Third Owner	17.70	1497.0	
3	Individual	Manual	First Owner	23.00	1396.0	
4	Individual	Manual	First Owner	16.10	1298.0	

	max_power	seats
0	74	5.0
1	103.52	5.0
2	78	5.0
3	90	5.0
4	88.2	5.0

0.1 Data Description

```
[3]: data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8128 entries, 0 to 8127
```

Data columns (total 12 columns):

#	Column	Non-Null Count	Dtype
0	name	8128 non-null	object
1	year	8128 non-null	int64
2	selling_price	8128 non-null	int64
3	km_driven	8128 non-null	int64
4	fuel	8128 non-null	object
5	seller_type	8128 non-null	object
6	transmission	8128 non-null	object
7	owner	8128 non-null	object
8	mileage(km/ltr/kg)	7907 non-null	float64
9	engine	7907 non-null	float64
10	max_power	7913 non-null	object
11	seats	7907 non-null	float64

dtypes: float64(3), int64(3), object(6)

memory usage: 762.1+ KB

```
[4]: data.describe().round(2)
```

```
[4]:
```

	year	selling_price	km_driven	mileage(km/ltr/kg)	engine	\
count	8128.00	8128.00	8128.00	7907.00	7907.00	
mean	2013.80	638271.81	69819.51	19.42	1458.63	
std	4.04	806253.40	56550.55	4.04	503.92	
min	1983.00	29999.00	1.00	0.00	624.00	
25%	2011.00	254999.00	35000.00	16.78	1197.00	
50%	2015.00	450000.00	60000.00	19.30	1248.00	
75%	2017.00	675000.00	98000.00	22.32	1582.00	
max	2020.00	1000000.00	2360457.00	42.00	3604.00	

	seats
count	7907.00
mean	5.42
std	0.96
min	2.00
25%	5.00
50%	5.00
75%	5.00
max	14.00

1 Data Preprocessing

Convert numeric Features to numeric datatype

```
[5]: data['max_power'] = pd.to_numeric(data['max_power'], errors='coerce')
```

Remove Unuseful Features

```
[6]: data = data.drop(columns='seller_type')
```

```
[7]: data.head()
```

```
[7]:
```

	name	year	selling_price	km_driven	fuel	\
0	Maruti Swift Dzire VDI	2014	450000	145500	Diesel	
1	Skoda Rapid 1.5 TDI Ambition	2014	370000	120000	Diesel	
2	Honda City 2017-2020 EXi	2006	158000	140000	Petrol	
3	Hyundai i20 Sportz Diesel	2010	225000	127000	Diesel	
4	Maruti Swift VXi BSIII	2007	130000	120000	Petrol	

	transmission	owner	mileage(km/ltr/kg)	engine	max_power	seats
0	Manual	First Owner	23.40	1248.0	74.00	5.0
1	Manual	Second Owner	21.14	1498.0	103.52	5.0
2	Manual	Third Owner	17.70	1497.0	78.00	5.0
3	Manual	First Owner	23.00	1396.0	90.00	5.0
4	Manual	First Owner	16.10	1298.0	88.20	5.0

```
[8]: data.shape
```

```
[8]: (8128, 11)
```

```
[ ]:
```

1.0.1 Check Nulls and remove them

```
[9]: data.isna().sum()
```

```
[9]: name                0
year                  0
selling_price         0
km_driven             0
fuel                 0
transmission          0
owner                 0
mileage(km/ltr/kg)    221
engine                221
max_power             216
seats                 221
dtype: int64
```

```
[10]: data = data.dropna()
```

```
[11]: data.isna().sum()
```

```
[11]: name                0
year                  0
```

```

selling_price      0
km_driven          0
fuel              0
transmission       0
owner             0
mileage(km/ltr/kg) 0
engine            0
max_power          0
seats             0
dtype: int64

```

1.0.2 Check Duplicates and remove them

```
[12]: data.duplicated().sum()
```

```
[12]: 1189
```

```
[13]: data = data.drop_duplicates()
```

```
[14]: data.duplicated().sum()
```

```
[14]: 0
```

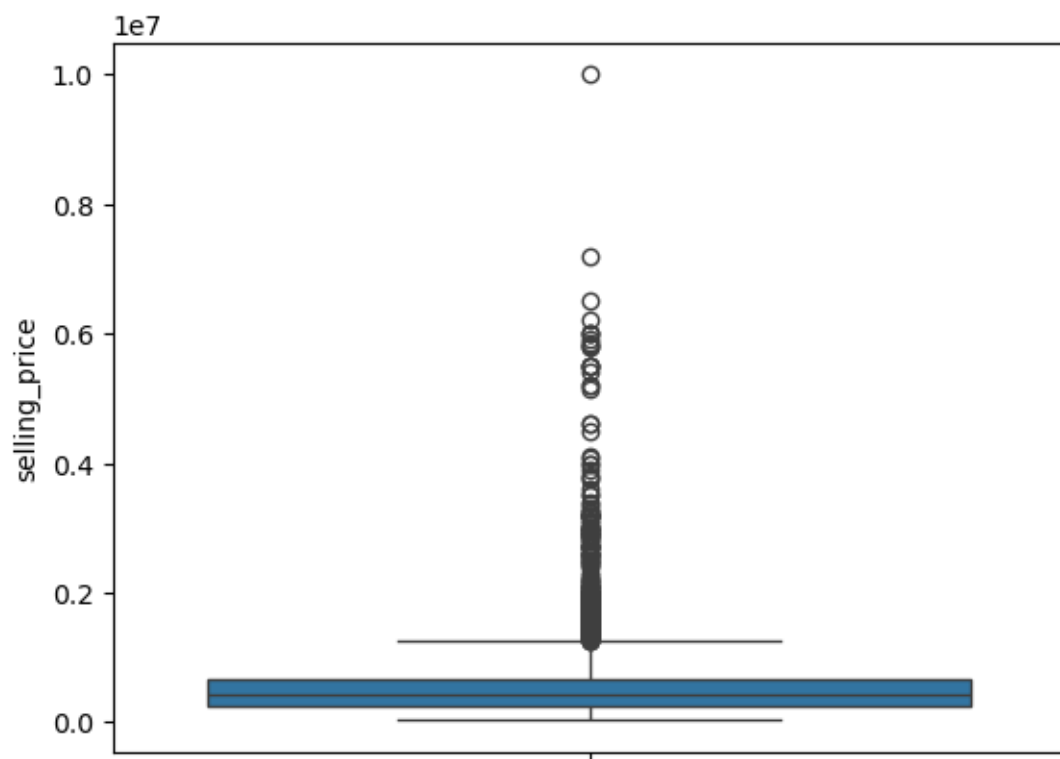
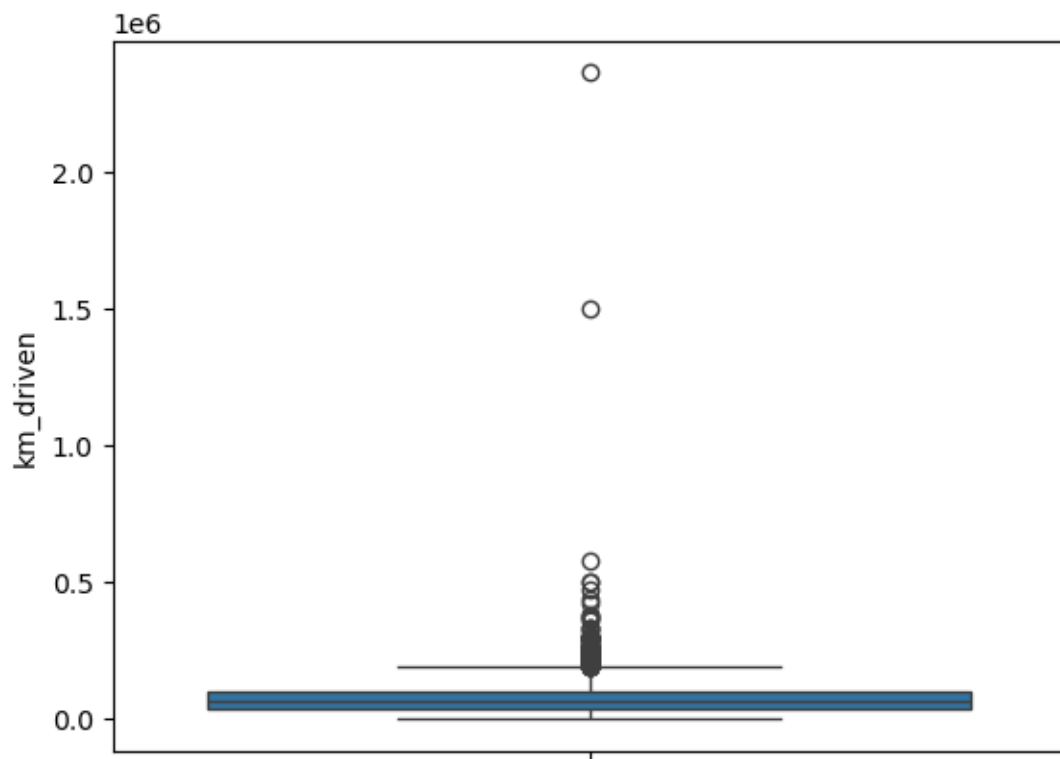
1.0.3 Check Outliers and remove them

```
[15]: print(data[['km_driven', 'selling_price']].describe().round(2))
```

	km_driven	selling_price
count	6717.00	6717.00
mean	73398.34	526386.00
std	58703.28	523550.45
min	1.00	29999.00
25%	38000.00	250000.00
50%	68203.00	420000.00
75%	100000.00	650000.00
max	2360457.00	10000000.00

```
[16]: sns.boxplot(data['km_driven'])
plt.show()

sns.boxplot(data['selling_price'])
plt.show()
```



```
[17]: # Calculate Q1, Q3 and IQR
```

```
Q1 = data[['km_driven', 'selling_price']].quantile(0.25)
```

```
Q3 = data[['km_driven', 'selling_price']].quantile(0.75)
```

```
IQR = Q3 - Q1
```

```
[18]: # Filter out outliers based on IQR
```

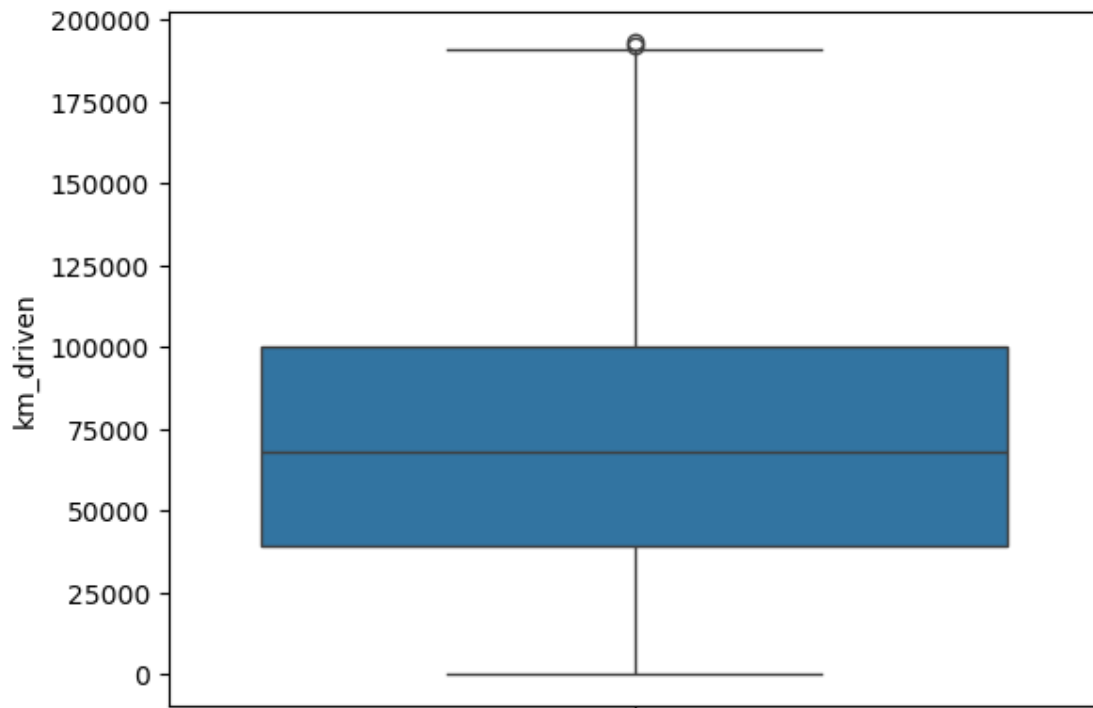
```
data = data[~((data[['km_driven', 'selling_price']] < (Q1 - 1.5 * IQR)) |  
→ (data[['km_driven', 'selling_price']] > (Q3 + 1.5 * IQR))).any(axis=1)]
```

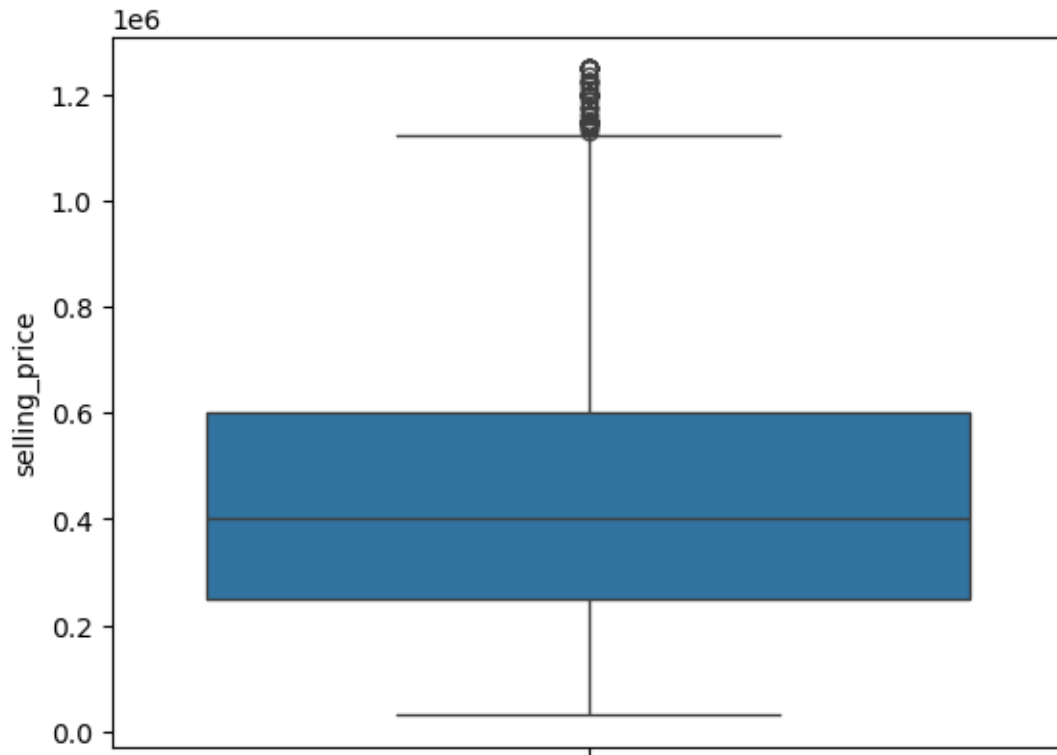
```
[19]: sns.boxplot(data['km_driven'])
```

```
plt.show()
```

```
sns.boxplot(data['selling_price'])
```

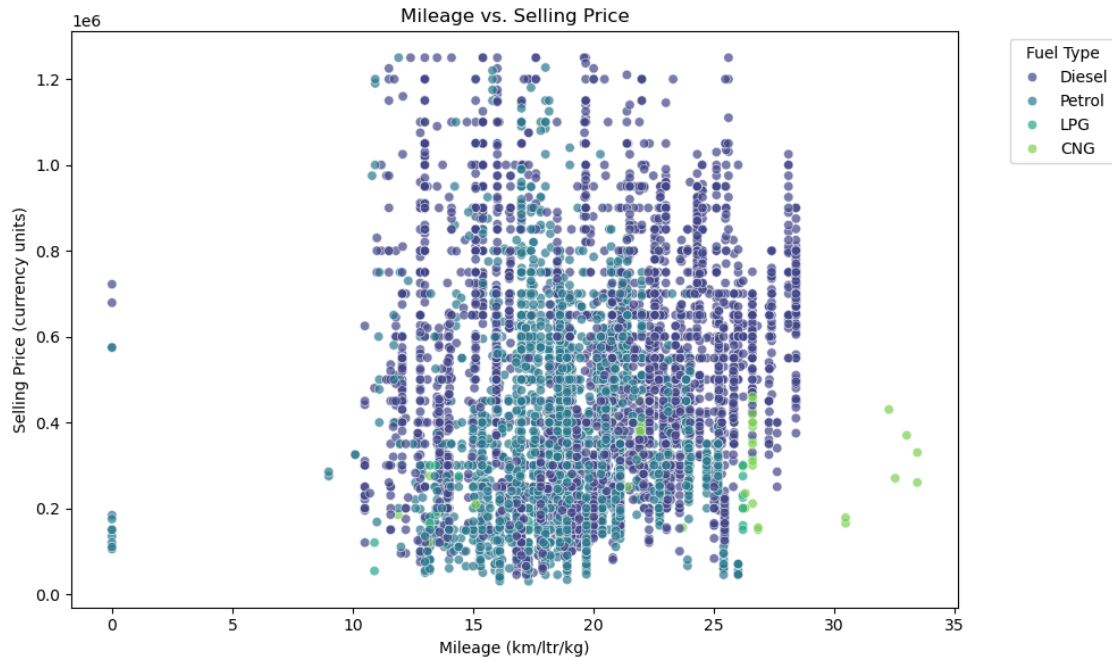
```
plt.show()
```





Visualization different charts

```
[20]: # Visualization 1: Scatter Plot - Mileage vs. Selling Price
plt.figure(figsize=(10, 6))
sns.scatterplot(
    data=data,
    x="mileage(km/ltr/kg)",
    y="selling_price",
    hue="fuel",
    palette="viridis",
    alpha=0.7
)
plt.title("Mileage vs. Selling Price")
plt.xlabel("Mileage (km/ltr/kg)")
plt.ylabel("Selling Price (currency units)")
plt.legend(title="Fuel Type", bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.show()
```



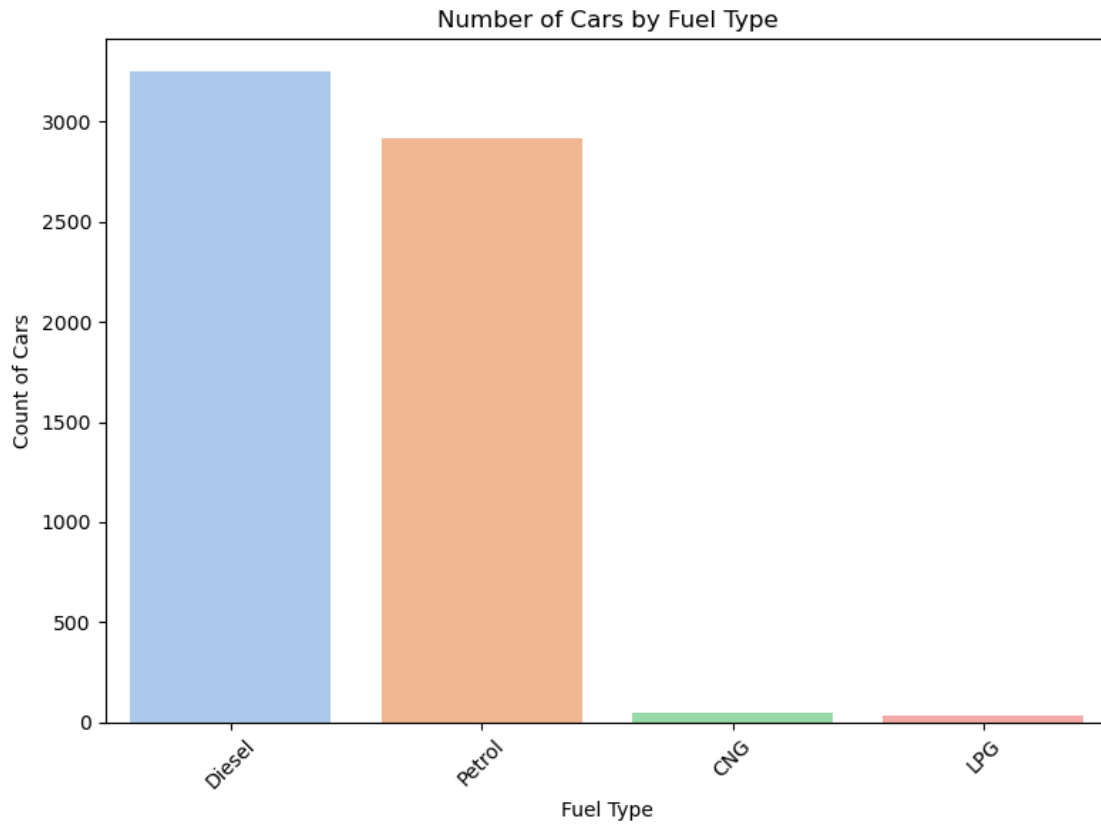
```
[21]: # Visualization 2: Bar Chart - Number of Cars by Fuel Type
plt.figure(figsize=(8, 6))
sns.countplot(data=data, x="fuel", palette="pastel", order=data["fuel"].
    ↳value_counts().index)
plt.title("Number of Cars by Fuel Type")
plt.xlabel("Fuel Type")
plt.ylabel("Count of Cars")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

C:\Users\Abdelrahman\AppData\Local\Temp\ipykernel_1856\980441756.py:3:

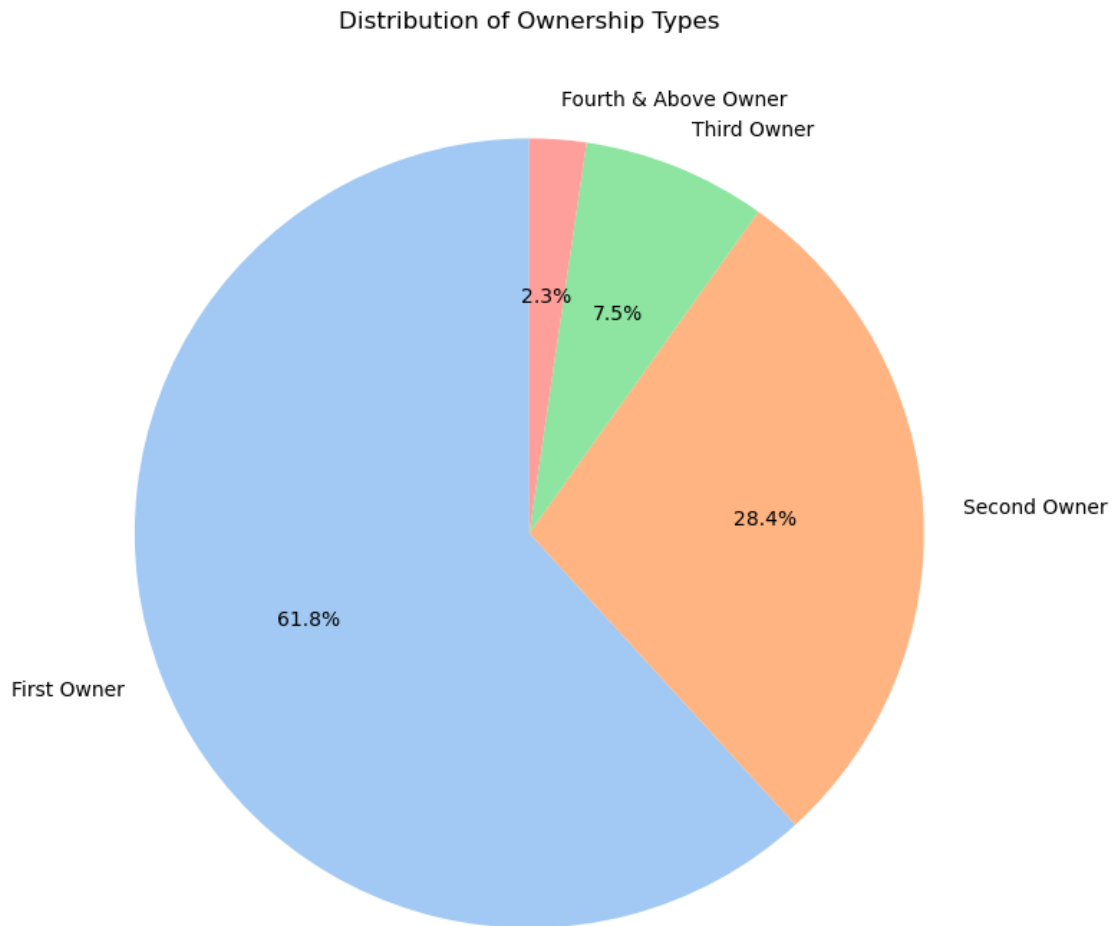
FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

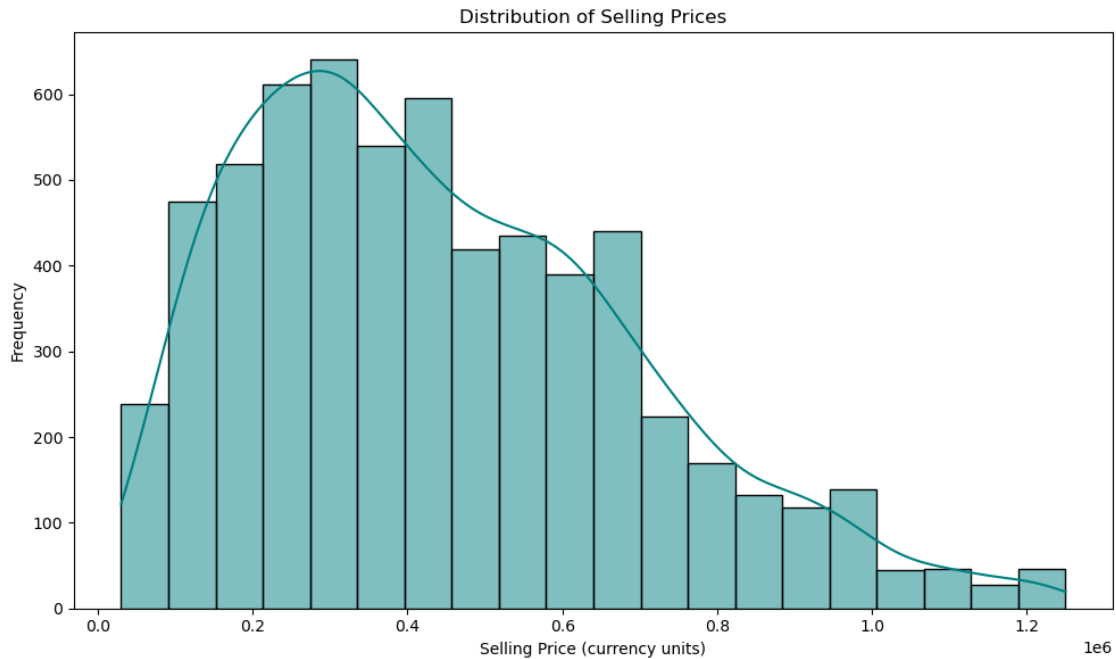
```
sns.countplot(data=data, x="fuel", palette="pastel",
order=data["fuel"].value_counts().index)
```



```
[22]: # Visualization 3: Pie Chart - Distribution of Ownership Types
plt.figure(figsize=(8, 8))
ownership_counts = data['owner'].value_counts()
plt.pie(
    ownership_counts,
    labels=ownership_counts.index,
    autopct='%1.1f%%',
    colors=sns.color_palette("pastel"),
    startangle=90
)
plt.title("Distribution of Ownership Types")
plt.tight_layout()
plt.show()
```



```
[23]: # Visualization 4: Histogram - Distribution of Selling Prices
plt.figure(figsize=(10, 6))
sns.histplot(data=data, x="selling_price", bins=20, kde=True, color="teal")
plt.title("Distribution of Selling Prices")
plt.xlabel("Selling Price (currency units)")
plt.ylabel("Frequency")
plt.tight_layout()
plt.show()
```

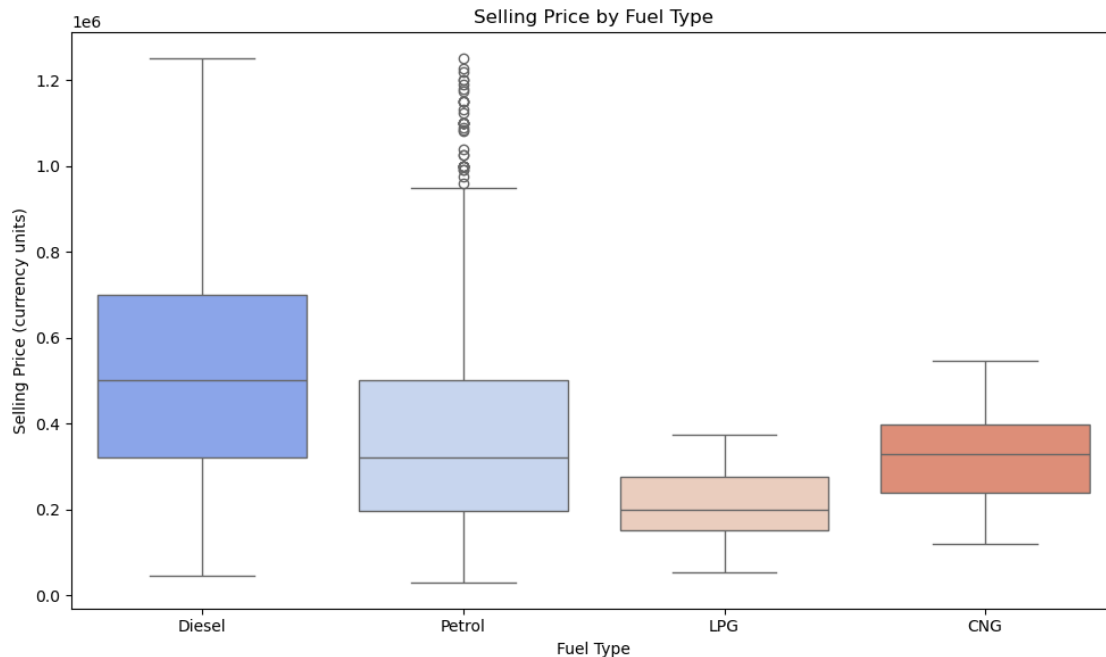


```
[24]: # Visualization 5: Box Plot - Selling Price by Fuel Type
plt.figure(figsize=(10, 6))
sns.boxplot(data=data, x="fuel", y="selling_price", palette="coolwarm")
plt.title("Selling Price by Fuel Type")
plt.xlabel("Fuel Type")
plt.ylabel("Selling Price (currency units)")
plt.tight_layout()
plt.show()
```

C:\Users\Abdelrahman\AppData\Local\Temp\ipykernel_1856\353594307.py:3:
FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.boxplot(data=data, x="fuel", y="selling_price", palette="coolwarm")
```



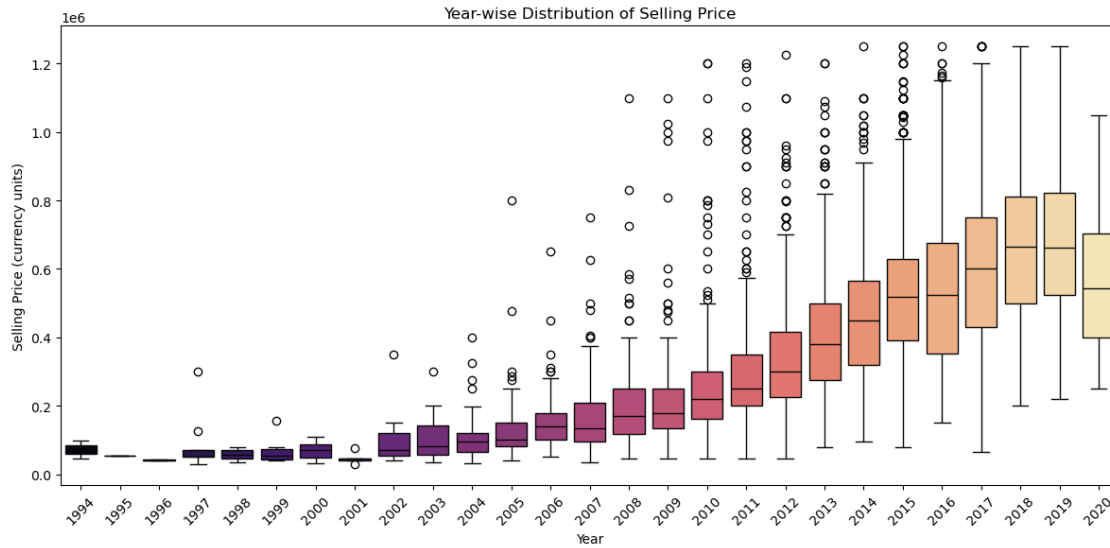
```
[25]: # 6 Year-wise Distribution of Selling Price
plt.figure(figsize=(12, 6))
sns.boxplot(data=data, x="year", y="selling_price", palette="magma")
plt.title("Year-wise Distribution of Selling Price")
plt.xlabel("Year")
plt.ylabel("Selling Price (currency units)")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

C:\Users\Abdelrahman\AppData\Local\Temp\ipykernel_1856\730013503.py:3:

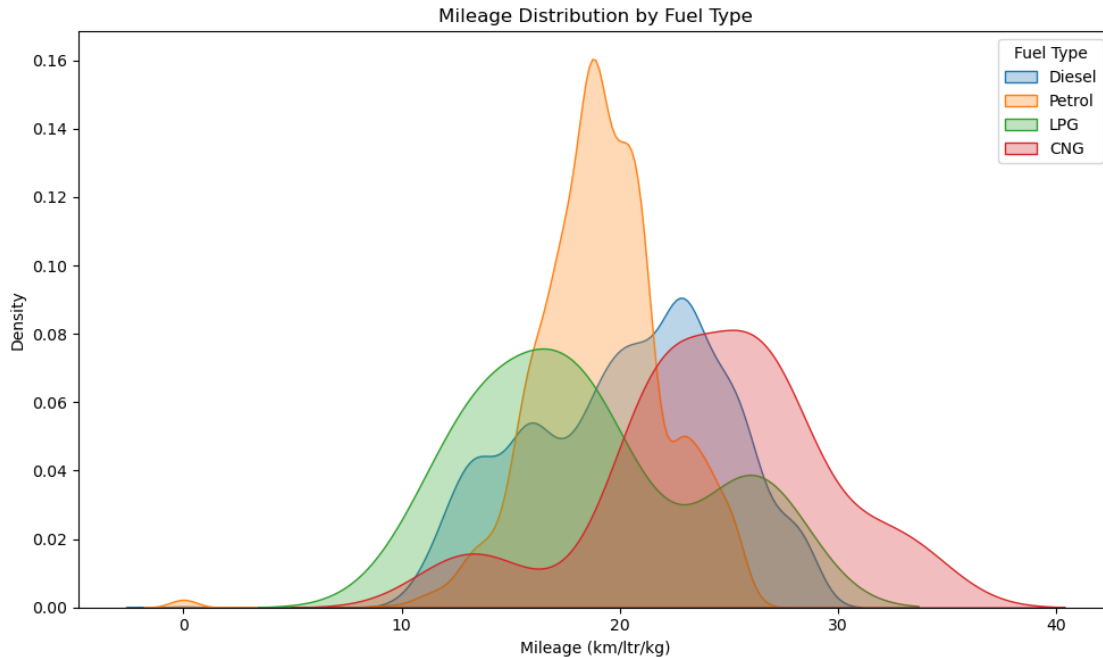
FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.boxplot(data=data, x="year", y="selling_price", palette="magma")
```



```
[26]: # 7 mileage Distribution by fuel type
plt.figure(figsize=(10, 6))
for fuel_type in data['fuel'].unique():
    sns.kdeplot(data[data['fuel'] == fuel_type]['mileage(km/ltr/kg)'],
                label=fuel_type, fill=True, alpha=0.3)
plt.title("Mileage Distribution by Fuel Type")
plt.xlabel("Mileage (km/ltr/kg)")
plt.ylabel("Density")
plt.legend(title="Fuel Type")
plt.tight_layout()
plt.show()
```



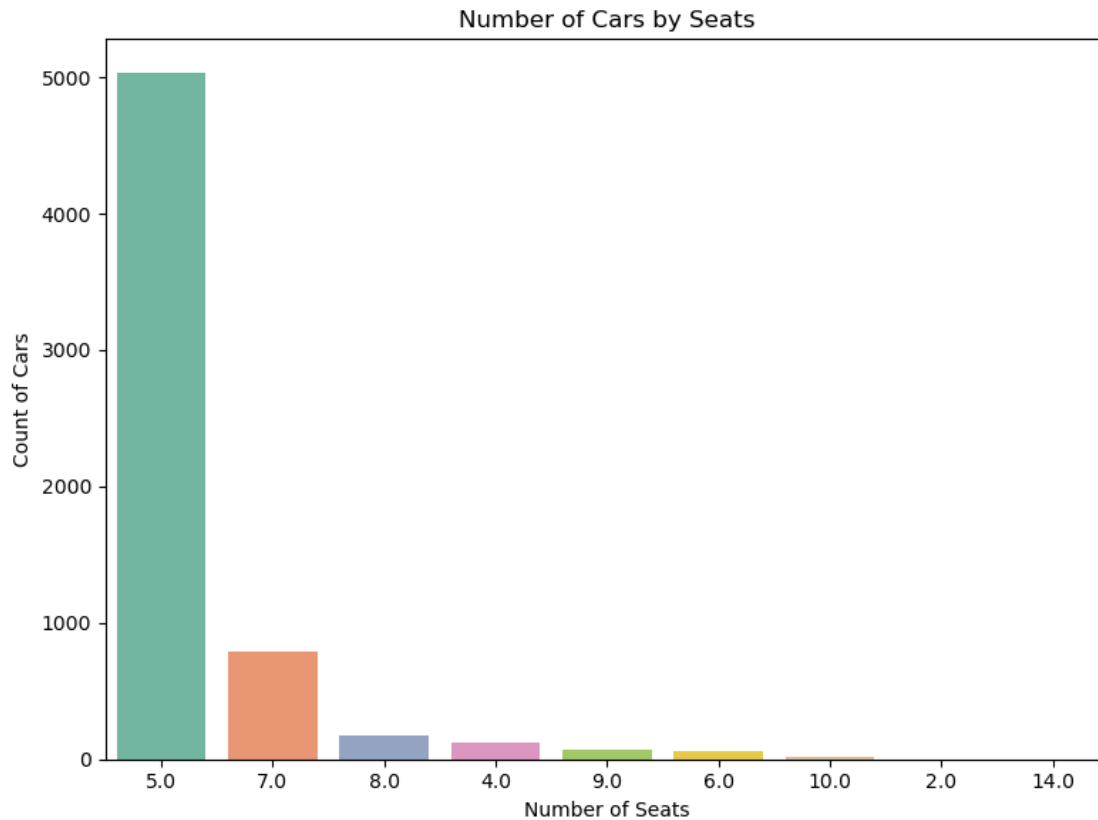
```
[27]: # 8 number of cars grouped by seats
plt.figure(figsize=(8, 6))
sns.countplot(data=data, x="seats", palette="Set2", order=data["seats"].
    ↳value_counts().index)
plt.title("Number of Cars by Seats")
plt.xlabel("Number of Seats")
plt.ylabel("Count of Cars")
plt.tight_layout()
plt.show()
```

C:\Users\Abdelrahman\AppData\Local\Temp\ipykernel_1856\1527416299.py:3:

FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.countplot(data=data, x="seats", palette="Set2",
order=data["seats"].value_counts().index)
```



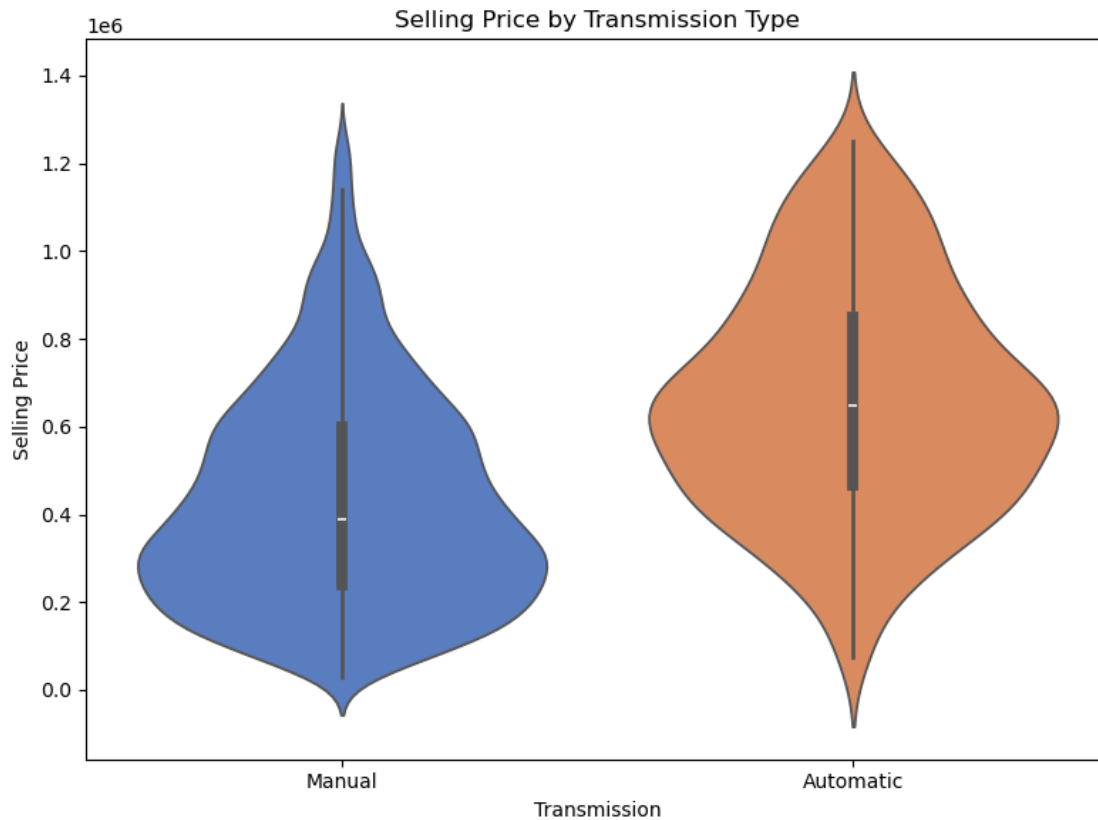
```
[28]: # 9 selling price by transmission type
plt.figure(figsize=(8, 6))
sns.violinplot(data=data, x="transmission", y="selling_price", palette="muted")
plt.title("Selling Price by Transmission Type")
plt.xlabel("Transmission")
plt.ylabel("Selling Price")
plt.tight_layout()
plt.show()
```

C:\Users\Abdelrahman\AppData\Local\Temp\ipykernel_1856\2204141520.py:3:

FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.violinplot(data=data, x="transmission", y="selling_price",
palette="muted")
```



1.1 Label Encoding.

```
[29]: data['owner'].value_counts()
```

```
[29]: owner
First Owner      3861
Second Owner     1773
Third Owner       471
Fourth & Above Owner  145
Name: count, dtype: int64
```

```
[30]: data['fuel'].value_counts()
```

```
[30]: fuel
Diesel    3250
Petrol    2916
CNG        50
LPG        34
Name: count, dtype: int64
```

```
[31]: data['transmission'].value_counts()
```

```
[31]: transmission
Manual      5874
Automatic    376
Name: count, dtype: int64
```

```
[32]: from sklearn.preprocessing import LabelEncoder
```

```
[33]: encode = LabelEncoder()
```

```
[34]: data_label = data.copy()
```

```
[35]: data_label['owner'] = encode.fit_transform(data_label['owner'])
data_label['fuel'] = encode.fit_transform(data_label['fuel'])
data_label['transmission'] = encode.fit_transform(data_label['transmission'])
```

```
[36]: data_label['owner'].value_counts() , data_label['fuel'].value_counts() ,
↳ data_label['transmission'].value_counts()
```

```
[36]: (owner
0      3861
2      1773
3       471
1       145
Name: count, dtype: int64,
fuel
1      3250
3      2916
0        50
2        34
Name: count, dtype: int64,
transmission
1      5874
0       376
Name: count, dtype: int64)
```

```
[37]: # data_label['owner'] = data_label['owner'].map({"First Owner":1, "Second
↳ Owner":2, "Third Owner":3, "Fourth & Above Owner":4})
# data_label['transmission'] = data_label['transmission'].map({"Manual":1,
↳ "Automatic":2})
# data_label['fuel'] = data_label['fuel'].map({"Diesel":1, "Petrol":2, "CNG":3,
↳ "LPG":4})

# OR
# pd.get_dummies(data_label['fuel'], prefix='fuel')
```

```
[ ]:
```

```
[38]: data_label.head()
```

```
[38]:
```

	name	year	selling_price	km_driven	fuel	\
0	Maruti Swift Dzire VDI	2014	450000	145500	1	
1	Skoda Rapid 1.5 TDI Ambition	2014	370000	120000	1	
2	Honda City 2017-2020 EXi	2006	158000	140000	3	
3	Hyundai i20 Sportz Diesel	2010	225000	127000	1	
4	Maruti Swift VXi BSIII	2007	130000	120000	3	

	transmission	owner	mileage(km/ltr/kg)	engine	max_power	seats
0	1	0	23.40	1248.0	74.00	5.0
1	1	2	21.14	1498.0	103.52	5.0
2	1	3	17.70	1497.0	78.00	5.0
3	1	0	23.00	1396.0	90.00	5.0
4	1	0	16.10	1298.0	88.20	5.0

```
[39]: data_label.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 6250 entries, 0 to 8125
Data columns (total 11 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   name                                  6250 non-null   object
1   year                                  6250 non-null   int64
2   selling_price                        6250 non-null   int64
3   km_driven                           6250 non-null   int64
4   fuel                                 6250 non-null   int32
5   transmission                         6250 non-null   int32
6   owner                                6250 non-null   int32
7   mileage(km/ltr/kg)                  6250 non-null   float64
8   engine                              6250 non-null   float64
9   max_power                           6250 non-null   float64
10  seats                               6250 non-null   float64
dtypes: float64(4), int32(3), int64(3), object(1)
memory usage: 641.7+ KB
```

```
[40]: data.describe().round(2)
```

```
[40]:
```

	year	selling_price	km_driven	mileage(km/ltr/kg)	engine	\
count	6250.00	6250.00	6250.00	6250.00	6250.00	
mean	2013.55	444491.00	69705.26	19.76	1375.91	
std	3.88	253225.24	39716.98	3.92	445.28	
min	1994.00	29999.00	1.00	0.00	624.00	
25%	2011.00	250000.00	39000.00	17.10	1196.00	

50%	2014.00	400000.00	68171.50	19.70	1248.00
75%	2017.00	600000.00	100000.00	22.71	1497.00
max	2020.00	1250000.00	193000.00	33.44	3498.00

	max_power	seats
count	6250.00	6250.00
mean	83.64	5.38
std	25.55	0.94
min	32.80	2.00
25%	67.10	5.00
50%	81.80	5.00
75%	97.70	5.00
max	272.00	14.00

```
[41]: data['brand'] = data['name'].apply(lambda x: str(x).split()[0])

brand_counts = data['brand'].value_counts()

most_sold_brand = brand_counts.idxmax()
most_sold_count = brand_counts.max()
print(f"most brand sold : {most_sold_brand} : {most_sold_count}")
print("\nmost 10 brand sold:")
print(brand_counts.head(20))
top_brands = brand_counts.head(20)
plt.figure(figsize=(10, 6))
top_brands.plot(kind='bar', color='skyblue')
plt.title("most 10 brand sold", fontsize=16)
plt.xlabel("brand", fontsize=12)
plt.ylabel("selles", fontsize=12)
plt.xticks(rotation=45)
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
data = data.drop(columns=['brand'])
```

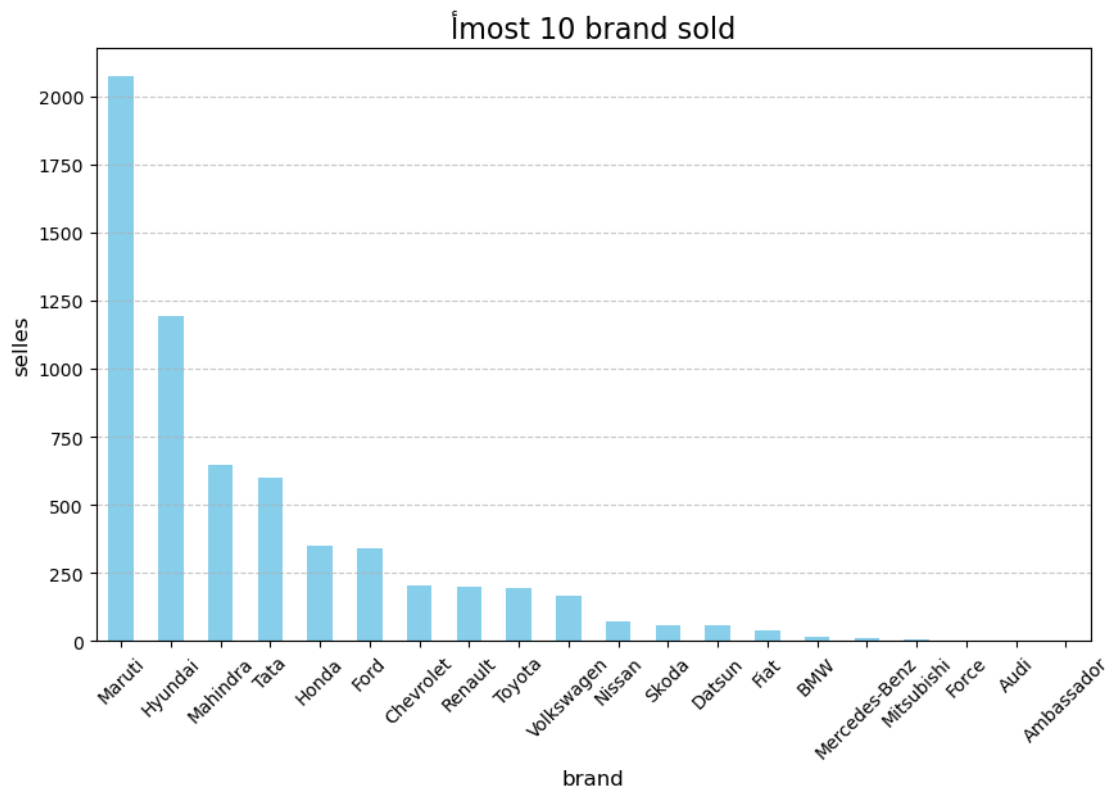
most brand sold : Maruti : 2072

most 10 brand sold:

brand	
Maruti	2072
Hyundai	1193
Mahindra	647
Tata	601
Honda	350
Ford	340
Chevrolet	206
Renault	202

Toyota	193
Volkswagen	167
Nissan	72
Skoda	60
Datsun	57
Fiat	38
BMW	14
Mercedes-Benz	11
Mitsubishi	7
Force	4
Audi	4
Ambassador	4

Name: count, dtype: int64



2 Clustering using K_Medoids

```
[61]: # pip install scikit-learn-extra
```

```
Collecting scikit-learn-extra
  Using cached scikit-learn-extra-0.3.0.tar.gz (818 kB)
  Installing build dependencies: started
```

```

Installing build dependencies: finished with status 'done'
Getting requirements to build wheel: started
Getting requirements to build wheel: finished with status 'done'
Preparing metadata (pyproject.toml): started
Preparing metadata (pyproject.toml): finished with status 'done'
Requirement already satisfied: numpy>=1.13.3 in
c:\users\abdelrahman\anaconda3\lib\site-packages (from scikit-learn-extra)
(1.26.4)
Requirement already satisfied: scipy>=0.19.1 in
c:\users\abdelrahman\anaconda3\lib\site-packages (from scikit-learn-extra)
(1.13.1)
Requirement already satisfied: scikit-learn>=0.23.0 in
c:\users\abdelrahman\anaconda3\lib\site-packages (from scikit-learn-extra)
(1.5.1)
Requirement already satisfied: joblib>=1.2.0 in
c:\users\abdelrahman\anaconda3\lib\site-packages (from scikit-
learn>=0.23.0->scikit-learn-extra) (1.4.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in
c:\users\abdelrahman\anaconda3\lib\site-packages (from scikit-
learn>=0.23.0->scikit-learn-extra) (3.5.0)
Building wheels for collected packages: scikit-learn-extra
  Building wheel for scikit-learn-extra (pyproject.toml): started
  Building wheel for scikit-learn-extra (pyproject.toml): finished with status
'done'
  Created wheel for scikit-learn-extra:
filename=scikit_learn_extra-0.3.0-cp312-cp312-win_amd64.whl size=402350
sha256=fb162b96f7b52d14ab7899d727dfde7426c2ef2749b0f06877923c0c25106015
  Stored in directory: c:\users\abdelrahman\appdata\local\pip\cache\wheels\17\4d
\c3\c6d5d563c1bf8146d059d63be3678abc2f2801fba0aaf5f0b8
Successfully built scikit-learn-extra
Installing collected packages: scikit-learn-extra
Successfully installed scikit-learn-extra-0.3.0
Note: you may need to restart the kernel to use updated packages.

```

```
[64]: from sklearn_extra.cluster import KMedoids
      from sklearn.preprocessing import StandardScaler
      import matplotlib.pyplot as plt
```

```
[65]: scaler = StandardScaler()
      data_clustering = scaler.fit_transform(data_label[['selling_price',
      ↪ 'max_power']])
```

```
[66]: kmedoids = KMedoids(n_clusters=3, random_state=42)
      labels = kmedoids.fit_predict(data_clustering)
      labels
```

```
[66]: array([2, 1, 0, ..., 0, 0, 0], dtype=int64)
```

```
[67]: data_label['luxuriousness_level'] = labels
```

```
[68]: medoid_indices = kmedoids.medoid_indices_
medoid_points = data_label.iloc[medoid_indices]
medoid_points
```

```
[68]:
```

	brand	model	year	selling_price	km_driven	\
5769	Hyundai	i10 Era	2013	245000	110000	
2218	Ford EcoSport 1.5 Petrol	Trend BSIV	2017	604000	30000	
117	Maruti	Swift Dzire VXI	2017	595000	40000	

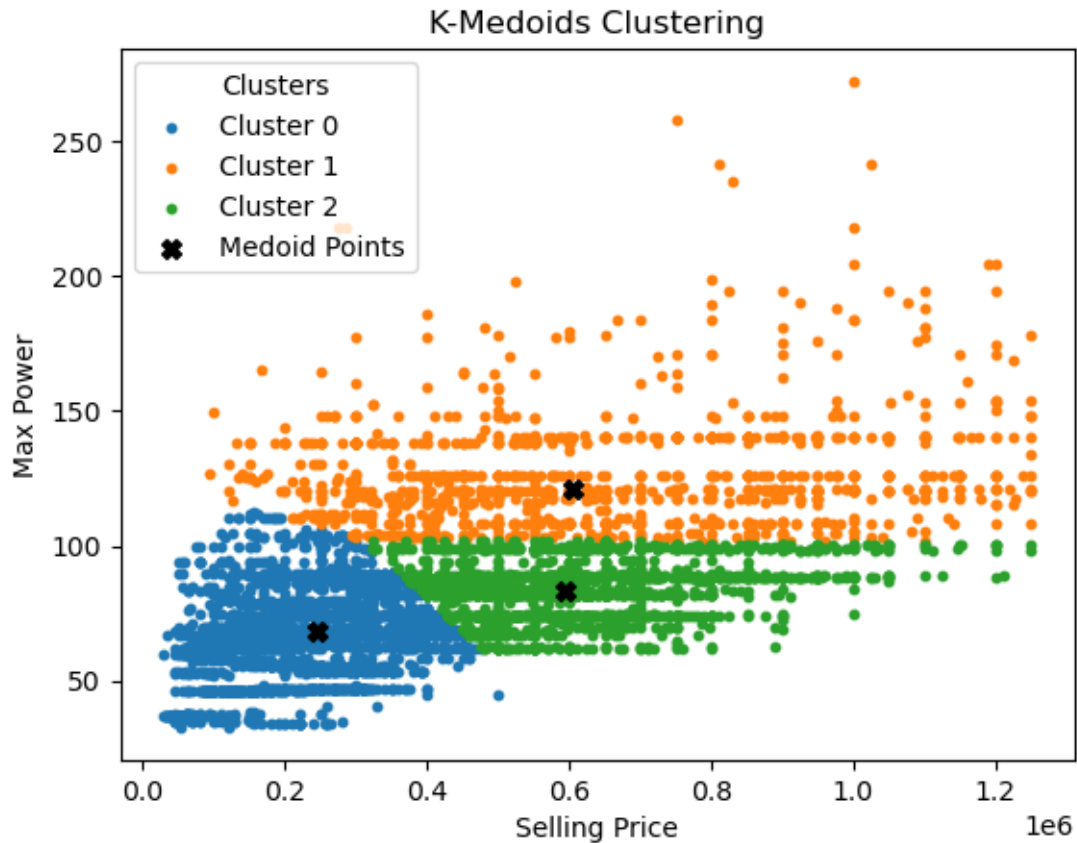
	fuel	transmission	owner	mileage(km/ltr/kg)	engine	max_power	seats	\
5769	3	1	2	19.81	1086.0	68.05	5.0	
2218	3	1	2	17.00	1497.0	121.36	5.0	
117	3	1	0	20.85	1197.0	83.14	5.0	

	luxuriousness_level
5769	0
2218	1
117	2

```
[69]: for cluster_label in range(kmedoids.n_clusters):
    cluster_data = data_label[data_label['luxuriousness_level'] ==
    ↪cluster_label]
    plt.scatter(
        cluster_data['selling_price'],
        cluster_data['max_power'],
        label=f'Cluster {cluster_label}',
        s=10
    )

plt.scatter(
    medoid_points['selling_price'],
    medoid_points['max_power'],
    color='black',
    label='Medoid Points',
    s=50,
    marker='X'
)

plt.xlabel('Selling Price')
plt.ylabel('Max Power')
plt.title('K-Medoids Clustering')
plt.legend(title='Clusters')
plt.show()
```



```
[70]: numeric_cols = data_label.select_dtypes(include=['float64', 'int64']).columns
encoded_col = data_label.select_dtypes(include=['int32']).columns
cluster_info = data_label.groupby('luxuriousness_level')[numeric_cols].mean()
cluster_info2 = data_label.groupby('luxuriousness_level')[encoded_col].
    ↪agg(lambda x: x.mode()[0])
cluster_info = pd.concat([cluster_info, cluster_info2], axis=1)
cluster_info['count'] = data_label.groupby('luxuriousness_level').size()
cluster_info
```

```
[70]:
```

	year	selling_price	km_driven \
luxuriousness_level			
0	2011.654798	241608.312309	75611.413700
1	2013.407136	648098.774931	79230.915828
2	2016.160779	614671.144475	57101.641304

	mileage(km/ltr/kg)	engine	max_power	seats \
luxuriousness_level				
0	19.752899	1180.457104	67.410926	5.134283
1	16.577127	1861.775846	126.103536	5.851784
2	21.358297	1396.434330	84.283265	5.480525

	luxuriousness_level	fuel	transmission	owner	count
0	0.0	3	1	0	2949
1	1.0	1	1	0	1093
2	2.0	1	1	0	2208

```
[71]: luxuriousness_levels = ['Economy', 'Comfort', 'Luxury']
data['luxuriousness_level'] = pd.Categorical(data_label['luxuriousness_level'],
categories=[0, 2, 1]).rename_categories(luxuriousness_levels)
```

```
[72]: data.head()
```

```
[72]:      brand      model  year  selling_price  km_driven  fuel  \
0  Maruti    Swift Dzire VDI  2014      450000    145500    1
1  Skoda  Rapid 1.5 TDI Ambition  2014      370000    120000    1
2  Honda    City 2017-2020 EXi  2006      158000    140000    3
3  Hyundai    i20 Sportz Diesel  2010      225000    127000    1
4  Maruti    Swift VXi BSIII  2007      130000    120000    3
```

	transmission	owner	mileage(km/ltr/kg)	engine	max_power	seats	\
0	1	0	23.40	1248.0	74.00	5.0	
1	1	2	21.14	1498.0	103.52	5.0	
2	1	3	17.70	1497.0	78.00	5.0	
3	1	0	23.00	1396.0	90.00	5.0	
4	1	0	16.10	1298.0	88.20	5.0	

	luxuriousness_level
0	Comfort
1	Luxury
2	Economy
3	Economy
4	Economy

```
[ ]:
```

2.1 Text Preprocessing

```
[42]: data[data['name'].str.contains('Honda', case=False, na=False)]['selling_price'].
shape
```

```
[42]: (350,)
```

```
[43]: data[data['name'] == 'Honda City 2017-2020 EXi']['selling_price'].shape
```

```
[43]: (7,)
```

```
[44]: data[(data['name'] == 'Honda City 2017-2020 EXi') & (data['year'] == 2006)][
    'selling_price'].shape
```

```
[44]: (2,)
```

```
[45]: data_label['name']
```

```
[45]: 0          Maruti Swift Dzire VDI
      1          Skoda Rapid 1.5 TDI Ambition
      2          Honda City 2017-2020 EXi
      3          Hyundai i20 Sportz Diesel
      4          Maruti Swift VXI BSIII
      ...
      8121      Maruti Wagon R VXI BS IV with ABS
      8122          Hyundai i20 Magna 1.4 CRDi
      8123          Hyundai i20 Magna
      8124          Hyundai Verna CRDi SX
      8125          Maruti Swift Dzire ZDi
      Name: name, Length: 6250, dtype: object
```

```
[46]: data_label['brand'] = data_label['name'].str.split(' ').str[0]
      # data_label['brand'] = data_label['name'].apply(lambda x: x.split()[0]) #
      # First word as Brand

      data_label['model'] = data_label['name'].apply(lambda x: " ".join(x.split()[1:
      ])) # Rest as Model
      data_label = data_label.drop(columns='name')
```

```
[47]: # Move 'brand' and 'model' to the first two columns
      cols = ['brand', 'model'] + [col for col in data_label.columns if col not in
      ['brand', 'model']]
      data_label = data_label[cols]
```

```
[48]: data_label
```

```
[48]:
```

	brand	model	year	selling_price	km_driven	\
0	Maruti	Swift Dzire VDI	2014	450000	145500	
1	Skoda	Rapid 1.5 TDI Ambition	2014	370000	120000	
2	Honda	City 2017-2020 EXi	2006	158000	140000	
3	Hyundai	i20 Sportz Diesel	2010	225000	127000	
4	Maruti	Swift VXI BSIII	2007	130000	120000	
...	...	...	...	...	...	
8121	Maruti	Wagon R VXI BS IV with ABS	2013	260000	50000	
8122	Hyundai	i20 Magna 1.4 CRDi	2014	475000	80000	
8123	Hyundai	i20 Magna	2013	320000	110000	
8124	Hyundai	Verna CRDi SX	2007	135000	119000	
8125	Maruti	Swift Dzire ZDi	2009	382000	120000	

	fuel	transmission	owner	mileage(km/ltr/kg)	engine	max_power	seats
0	1	1	0	23.40	1248.0	74.00	5.0
1	1	1	2	21.14	1498.0	103.52	5.0
2	3	1	3	17.70	1497.0	78.00	5.0
3	1	1	0	23.00	1396.0	90.00	5.0
4	3	1	0	16.10	1298.0	88.20	5.0
...	...	...	...	...	...	...	...
8121	3	1	2	18.90	998.0	67.10	5.0
8122	1	1	2	22.54	1396.0	88.73	5.0
8123	3	1	0	18.50	1197.0	82.85	5.0
8124	1	1	1	16.80	1493.0	110.00	5.0
8125	1	1	0	19.30	1248.0	73.90	5.0

[6250 rows x 12 columns]

```
[49]: data_prep = data_label.copy()
```

2.1.1 one-hot encoding

```
[50]: # Apply One-Hot Encoding for 'brand' and 'model'
data_prep = pd.get_dummies(data_prep, columns=['brand', 'model'],
                             drop_first=True)
```

```
[51]: # Track the columns after one-hot encoding (used for alignment)
one_hot_columns = data_prep.columns.difference(['selling_price'])
```

```
[52]: data_prep
```

```
[52]:
```

	year	selling_price	km_driven	fuel	transmission	owner	\
0	2014	450000	145500	1	1	0	
1	2014	370000	120000	1	1	2	
2	2006	158000	140000	3	1	3	
3	2010	225000	127000	1	1	0	
4	2007	130000	120000	3	1	0	
...	...	...	...	...	...	...	...
8121	2013	260000	50000	3	1	2	
8122	2014	475000	80000	1	1	2	
8123	2013	320000	110000	3	1	0	
8124	2007	135000	119000	1	1	1	
8125	2009	382000	120000	1	1	0	

	mileage(km/ltr/kg)	engine	max_power	seats	...	\
0	23.40	1248.0	74.00	5.0	...	
1	21.14	1498.0	103.52	5.0	...	
2	17.70	1497.0	78.00	5.0	...	
3	23.00	1396.0	90.00	5.0	...	

4	16.10	1298.0	88.20	5.0	...
...	...	...	...	...	...
8121	18.90	998.0	67.10	5.0	...
8122	22.54	1396.0	88.73	5.0	...
8123	18.50	1197.0	82.85	5.0	...
8124	16.80	1493.0	110.00	5.0	...
8125	19.30	1248.0	73.90	5.0	...

	model_i20 Magna 1.4 CRDi (Diesel)	model_i20 Magna Optional 1.2 \
0	False	False
1	False	False
2	False	False
3	False	False
4	False	False
...	...	...
8121	False	False
8122	False	False
8123	False	False
8124	False	False
8125	False	False

	model_i20 Magna Optional 1.4 CRDi	model_i20 Petrol CVT Asta \
0	False	False
1	False	False
2	False	False
3	False	False
4	False	False
...	...	...
8121	False	False
8122	False	False
8123	False	False
8124	False	False
8125	False	False

	model_i20 Sportz 1.2	model_i20 Sportz 1.4 CRDi \
0	False	False
1	False	False
2	False	False
3	False	False
4	False	False
...	...	...
8121	False	False
8122	False	False
8123	False	False
8124	False	False
8125	False	False

	model_i20 Sportz AT 1.4	model_i20 Sportz Diesel \
0	False	False
1	False	False
2	False	False
3	False	True
4	False	False
...	...	...
8121	False	False
8122	False	False
8123	False	False
8124	False	False
8125	False	False

	model_i20 Sportz Option 1.2	model_i20 Sportz Option 1.4 CRDi
0	False	False
1	False	False
2	False	False
3	False	False
4	False	False
...	...	...
8121	False	False
8122	False	False
8123	False	False
8124	False	False
8125	False	False

[6250 rows x 1831 columns]

Note: Many machine learning models (like tree-based algorithms such as Random Forest or Gradient Boosting) can handle high-dimensional datasets and one-hot encoded features effectively without dimensionality reduction. If the features are meaningful and predictive (e.g., specific car models or brands), PCA might reduce accuracy because it loses direct relationships between features and the target.

3 Machine Learning

To create a machine learning model that predicts the selling price of a car based on the car's model, brand and year**, and then provides similar car specifications

3.1 Model Building

Use Random Forest Regression

3.1.1 Split data

```
[73]: from sklearn.model_selection import train_test_split

      # Features and target
      X = data_prep[one_hot_columns] # Use the one-hot encoded columns
      y = data_prep['selling_price'] # Target variable

[74]: # Split the dataset into train and test sets
      X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
      ↪random_state=42)
```

3.2 Standardization

```
[75]: from sklearn.preprocessing import StandardScaler

      # Fit the scaler on training data
      scaler = StandardScaler()
      # scaler = StandardScaler().fit(X_train)

[76]: X_train_scaled = scaler.fit_transform(X_train) # Scale training data
      X_test_scaled = scaler.fit_transform(X_test)  # Scale test data using the same
      ↪scaler
```

3.2.1 Model Training

```
[77]: from sklearn.ensemble import RandomForestRegressor

      # Initialize the RandomForestRegressor model
      model = RandomForestRegressor(n_estimators=100, random_state=42)

      # Train the model
      model.fit(X_train_scaled, y_train)
```

```
[77]: RandomForestRegressor(random_state=42)
```

```
[78]: # Make predictions
      y_pred = model.predict(X_test_scaled)
      y_pred
```

```
[78]: array([309444.68      , 491839.98      , 398476.65666667, ...,
      137966.64333333, 453431.66666667, 407289.97      ])
```

```
[79]: y_test
```

```
[79]: 6459    299000
      746    450000
```

```

7156    440000
5492    280000
5890    150000
...
2053    380000
1889    120000
695     135000
584     500000
7247    770000
Name: selling_price, Length: 1250, dtype: int64

```

3.3 Model Evaluation

```

[80]: from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# Evaluate the model using MAE, MSE, and R2 score
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

```

```

[81]: # Print the evaluation metrics
print(f'Mean Absolute Error (MAE): {mae}')
print(f'Mean Squared Error (MSE): {mse}')
print(f'R2 Score: {r2}')

```

```

Mean Absolute Error (MAE): 62975.423092431745
Mean Squared Error (MSE): 7611362815.672786
R2 Score: 0.8807675722548839

```

3.4 Validation (Tuning Hyperparameters)

it will take almost 15 minutes

```

[82]: from sklearn.model_selection import GridSearchCV

# Define hyperparameters to tune
param_grid = {
    'n_estimators': [100, 200],
    'max_depth': [10, 20, 30],
    'min_samples_split': [2, 5],
}

```

```

[83]: # # Grid Search
# grid_search = GridSearchCV(estimator=RandomForestRegressor(random_state=42),
#                             param_grid=param_grid, cv=3)
# grid_search.fit(X_train, y_train)

```

```
[84]: # # Best parameters and model
# print(f'Best parameters: {grid_search.best_params_}')
# best_model = grid_search.best_estimator_
# print(best_model)
```

3.5 Retrain Model

```
[85]: # Train the model with the best parameters
model = RandomForestRegressor(max_depth=30, min_samples_split=5,
    ↪n_estimators=200, random_state=42)
model.fit(X_train_scaled, y_train)
```

```
[85]: RandomForestRegressor(max_depth=30, min_samples_split=5, n_estimators=200,
    random_state=42)
```

```
[86]: # Predictions on test data
y_test_pred = model.predict(X_test_scaled)

# Calculate the performance on test data
print("\nTest Performance:")
print(f"Mean Absolute Error: {mean_absolute_error(y_test, y_test_pred)}")
print(f"Mean Squared Error: {mean_squared_error(y_test, y_test_pred)}")
print(f"R2 Score: {r2_score(y_test, y_test_pred)}")
```

Test Performance:
Mean Absolute Error: 62728.47035199182
Mean Squared Error: 7573206954.301268
R2 Score: 0.8813652859750425

3.6 Model Testing

Price prediction

```
[87]: def preprocess_and_predict(new_data):
    # Manual mapping for categorical variables
    fuel_mapping = {'Diesel': 1, 'Petrol': 3, 'CNG': 0, 'LPG': 2}
    transmission_mapping = {'Manual': 1, 'Automatic': 0}
    owner_mapping = {
        'First Owner': 0,
        'Second Owner': 2,
        'Third Owner': 3,
        'Fourth & Above Owner': 1
    }

    # Apply the mappings
    if 'fuel' in new_data:
        new_data['fuel'] = new_data['fuel'].map(fuel_mapping)
```

```

    if 'transmission' in new_data:
        new_data['transmission'] = new_data['transmission'].
↪map(transmission_mapping)
    if 'owner' in new_data:
        new_data['owner'] = new_data['owner'].map(owner_mapping)

    # Handle missing values after mapping
    new_data = new_data.fillna(0)

    # Split 'name' into 'brand' and 'model'
    if 'name' in new_data:
        new_data['brand'] = new_data['name'].str.split(' ').str[0]
        new_data['model'] = new_data['name'].apply(lambda x: " ".join(x.
↪split()[1:]))
        new_data = new_data.drop(columns='name')

    # Apply one-hot encoding
    new_data_encoded = pd.get_dummies(new_data, columns=['brand', 'model'],
↪drop_first=True)

    # Align columns with training data (handle missing or extra columns)
    new_data_aligned = pd.DataFrame(0, columns=one_hot_columns,
↪index=new_data_encoded.index) # Create a DataFrame with all one-hot columns
    new_data_aligned.update(new_data_encoded) # Update with the
↪new_data_encoded

    # Scale the data
    scaled_data = scaler.transform(new_data_aligned)

    # Predict using the model
    predictions = model.predict(scaled_data)

    return predictions

```

```

[88]: # Example new data
new_data = pd.DataFrame({
    'name': ['Maruti Swift Dzire VDI', 'Hyundai i20 Sportz Diesel'],
    'year': [2014, 2010],
    'km_driven': [145500, 127000],
    'fuel': ['Diesel', 'Diesel'],
    'transmission': ['Manual', 'Manual'],
    'owner': ['First Owner', 'First Owner'],
    'mileage(km/ltr/kg)': [23.4, 23.0],
    'engine': [1248.0, 1396.0],
    'max_power': [74.0, 90.0],
    'seats': [5.0, 5.0]
})

```

```
# Make predictions
predictions = preprocess_and_predict(new_data)

print("Predictions:", predictions)
```

Predictions: [491502.65456999 299361.60617366]

C:\Users\Abdelrahman\AppData\Local\Temp\ipykernel_1856\3107821965.py:34:
FutureWarning: Setting an item of incompatible dtype is deprecated and will raise in a future error of pandas. Value '[True False]' has dtype incompatible with int64, please explicitly cast to a compatible dtype first.

```
new_data_aligned.update(new_data_encoded) # Update with the new_data_encoded
```

C:\Users\Abdelrahman\AppData\Local\Temp\ipykernel_1856\3107821965.py:34:
FutureWarning: Setting an item of incompatible dtype is deprecated and will raise in a future error of pandas. Value '[23.4 23.]' has dtype incompatible with int64, please explicitly cast to a compatible dtype first.

```
new_data_aligned.update(new_data_encoded) # Update with the new_data_encoded
```

C:\Users\Abdelrahman\AppData\Local\Temp\ipykernel_1856\3107821965.py:34:
FutureWarning: Setting an item of incompatible dtype is deprecated and will raise in a future error of pandas. Value '[False True]' has dtype incompatible with int64, please explicitly cast to a compatible dtype first.

```
new_data_aligned.update(new_data_encoded) # Update with the new_data_encoded
```

[]:

3.7 Exactly Searching

```
[89]: def search_car(name, year):
      # Filter data based on brand, model, and year
      filtered_data = data[(data['name'] == name) & (data['year'] == year)]

      return filtered_data
```

```
[90]: # Example usage
name_input = 'Hyundai i20 Sportz Diesel'
year_input = 2010

result = search_car(name_input, year_input)
result
```

```
[90]:
```

	name	year	selling_price	km_driven	fuel	\
3	Hyundai i20 Sportz Diesel	2010	225000	127000	Diesel	

	transmission	owner	mileage(km/ltr/kg)	engine	max_power	seats
3	Manual	First Owner	23.0	1396.0	90.0	5.0

3.8 Predicted Searching

```
[91]: #!pip install fuzzywuzzy
```

```
[92]: from fuzzywuzzy import fuzz

def find_top_matches(name, year, top_n=3):
    # Create a list to store the scores and corresponding rows
    match_scores = []

    for index, row in data.iterrows():
        # Calculate the similarity for name
        name_score = fuzz.ratio(name.lower(), row['name'].lower())

        # Calculate the difference in year (lower difference is better)
        year_score = 1 / (1 + abs(row['year'] - year)) # Inverse of year
        ↪ difference

        # Combine scores (you can adjust weights as needed)
        combined_score = (name_score * 0.4) + (year_score * 0.2)

        # Append the combined score and the row to the list
        match_scores.append((combined_score, row))

    # Sort the matches by score in descending order (best match first)
    match_scores.sort(reverse=True, key=lambda x: x[0])

    # Return the top_n best matches
    top_matches = [match[1] for match in match_scores[:top_n]]

    return top_matches
```

```
C:\Users\Abdelrahman\anaconda3\Lib\site-packages\fuzzywuzzy\fuzz.py:11:
UserWarning: Using slow pure-python SequenceMatcher. Install python-Levenshtein
to remove this warning
  warnings.warn('Using slow pure-python SequenceMatcher. Install python-
Levenshtein to remove this warning')
```

```
[93]: # Example usage
name_input = 'Hyundai i20 Magna'
year_input = 2012

# Find the top 3 nearest matches
top_matches = find_top_matches(name_input, year_input)

# Display the results
print("Top Matches Found:")
```

```

print('-----')
for match in top_matches:
    print(match)
    print()

```

Top Matches Found:

```

name                Hyundai i20 Magna
year                2012
selling_price       335000
km_driven           45000
fuel                Petrol
transmission        Manual
owner               First Owner
mileage(km/ltr/kg)  18.5
engine              1197.0
max_power           82.85
seats               5.0
Name: 7021, dtype: object

```

```

name                Hyundai i20 Magna
year                2013
selling_price       320000
km_driven           31377
fuel                Petrol
transmission        Manual
owner               Second Owner
mileage(km/ltr/kg)  18.5
engine              1197.0
max_power           82.85
seats               5.0
Name: 213, dtype: object

```

```

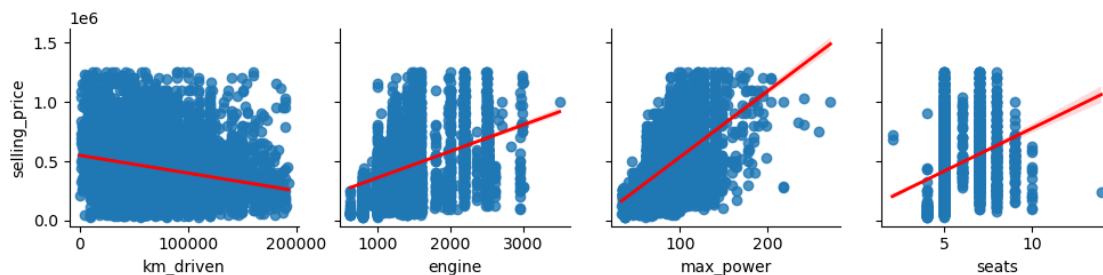
name                Hyundai i20 Magna
year                2013
selling_price       350000
km_driven           90000
fuel                Petrol
transmission        Manual
owner               Second Owner
mileage(km/ltr/kg)  18.5
engine              1197.0
max_power           82.85
seats               5.0
Name: 2792, dtype: object

```

```
[94]: # # Display the results
# print("Top Matches Found:")
# print('-----')
# for match in top_matches:
#     print(match[['name', 'year', 'selling_price']]) # Display the name,
#     ↪ year, and price columns
#     print()
```

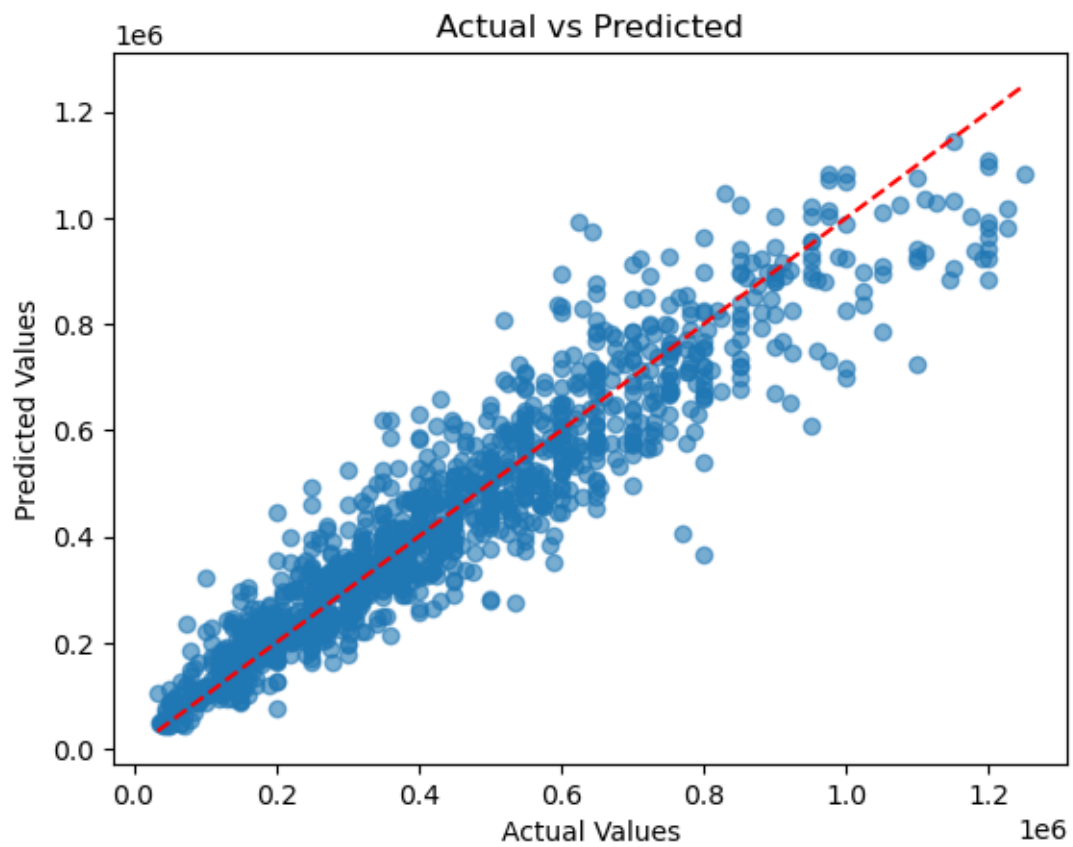
```
[95]: import seaborn as sns
import matplotlib.pyplot as plt
sns.pairplot(data_prep, y_vars='selling_price', x_vars=['km_driven', 'engine',
↪ 'max_power', 'seats'], kind='reg', plot_kws={'line_kws':{'color':'red',
↪ 'linewidth':2}})
```

[95]: <seaborn.axisgrid.PairGrid at 0x2e103c34aa0>



```
[96]: import matplotlib.pyplot as plt

plt.scatter(y_test, y_pred, alpha=0.6)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()],
↪ color='red', linestyle='--')
plt.xlabel('Actual Values')
plt.ylabel('Predicted Values')
plt.title('Actual vs Predicted')
plt.show()
```



[]: